立创·庐山派 K230-CanMV 开发板 API 手册总结

> 本文档基于 立创开发板技术文档中心 编写，系统总结了庐山派 K230-CanMV 开发板的各模块函数和类的功能、参数及用法。

目录

1. 概述
2. 基础外设模块 (Machine)
    - 2.1 Pin - GPIO引脚控制
    - 2.2 FPIOA - 引脚功能复用
    - 2.3 PWM - 脉冲宽度调制
    - 2.4 UART - 串口通信
    - 2.5 ADC - 模数转换
    - 2.6 RTC - 实时时钟
    - 2.7 Timer - 定时器
    - 2.8 WDT - 看门狗
    - 2.9 I2C - I2C通信
    - 2.10 SPI - SPI通信
    - 2.11 TOUCH - 触摸屏
    - 2.12 LED - LED控制
3. 多媒体模块 (MPP)
    - 3.1 Sensor - 摄像头传感器
    - 3.2 Display - 图像显示
    - 3.3 Audio - 音频模块
    - 3.4 MediaManager - 媒体管理器
    - 3.5 VDEC - 视频解码
    - 3.6 VENC - 视频编码
    - 3.7 MP4 - 多媒体封装
    - 3.8 Player - 播放器
    - 3.9 RTSP - 实时流媒体
    - 3.10 PM - 电源管理
4. 图像处理模块 (OpenMV兼容)
5. 网络模块
    - 5.1 network - 网络配置
    - 5.2 socket - 网络通信
6. AI Demo 模块
    - 6.1 PipeLine - AI流程管道
    - 6.2 Ai2d - AI预处理
    - 6.3 AIBase - AI推理基类
    - 6.4 YOLO - 目标检测
    - 6.5 nncase_runtime - 神经网络运行时
7. 标准库与扩展库
8. 完整例程速查

1. 概述

立创·庐山派 K230-CanMV 是基于嘉楠科技 K230 芯片的开发板，支持 MicroPython 开发环境 (CanMV)。K230 芯片具有以下特性：

- **双核处理器**：大核 RISC-V 1.6GHz + 小核 RISC-V 0.8GHz
- **KPU (神经网络处理器)**：支持 AI 推理加速
- **64 个 GPIO 引脚**，通过 FPIOA 灵活配置功能
- **多媒体处理**：支持 3 路摄像头输入、多通道视频输出
- **丰富外设**：UART、I2C、SPI、PWM、ADC、Timer、WDT、RTC 等

所有硬件模块均可通过 `machine` 模块访问，多媒体功能通过 `media` 和 `mpp` 模块访问，AI 功能通过 `libs` 模块中的 AIBase、PipeLine 等类访问。

---

2. 基础外设模块 (Machine)

2.1 Pin - GPIO引脚控制

K230 芯片内部包含 **64 个 GPIO 引脚**（编号 0-63），每个引脚可独立配置为输入或输出模式，支持上下拉电阻和驱动能力设置。

类： `machine.Pin`

方法	语法	说明
构造函数	`Pin(index, mode, pull=PULL_NONE, drive=7)`	`index`: 0-63; `mode`: `Pin.OUT`/`Pin.IN`; `pull`: `PULL_UP`/`PULL_DOWN`/`PULL_NONE`; `drive`: 驱动能力 0-7
init	`Pin.init(mode, pull=PULL_NONE, drive=7)`	初始化引脚模式、上下拉配置及驱动能力，无返回值
value	`Pin.value([value])`	获取引脚输入电平（无参数）或设置输出电平（传0/1）
mode	`Pin.mode([mode])`	获取或设置引脚模式（`Pin.IN`/`Pin.OUT`），返回或设置后无返回值
pull	`Pin.pull([pull])`	获取或设置上下拉配置
drive	`Pin.drive([drive])`	获取或设置驱动能力 0-7
on	`Pin.on()`	输出高电平，无参数无返回值
off	`Pin.off()`	输出低电平，无参数无返回值
high	`Pin.high()`	等同于 on()
low	`Pin.low()`	等同于 off()

```
from machine import Pin
pin = Pin(2, Pin.OUT, pull=Pin.PULL_NONE, drive=7)
pin.value(1)          # 输出高电平
print(pin.value())    # 读取电平值
```

2.2 FPIOA - 引脚功能复用

FPIOA 负责配置物理引脚的功能。多个功能共享同一 I/O 引脚，通过 FPIOA 选择激活的功能。

类： `machine.FPIOA`

方法	语法	说明
构造函数	`fpioa = FPIOA()`	实例化，无参数

方法	语法	说明
set_function	`FPIOA.set_function(pin, func, ie=-1, oe=-1, pu=-1, pd=-1, st=-1, ds=-1)`	设置引脚功能。`pin`: 0-63; `func`: 功能号; `ie/oe`: 输入/输出使能; `pu/pd`: 上/下拉; `st`: 施密特; `ds`: 驱动0-7。传-1表示不修改此参数
get_pin_num	`fpioa.get_pin_num(func)`	获取指定功能当前所在的引脚号，未找到返回 None
get_pin_func	`fpioa.get_pin_func(pin)`	获取指定引脚当前的功能号
help	`fpioa.help([number, func=False])`	打印配置提示。无参数打印所有引脚; 传入引脚号打印该引脚详情; func=True且传入功能号打印该功能可用引脚

```
from machine import FPIOA
fpioa = FPIOA()
fpioa.set_function(0, FPIOA.GPIO0)             # 设置引脚0为GPIO0
fpioa.set_function(2, FPIOA.GPIO2, ie=1, oe=1, pu=0, pd=0, st=1, ds=7)
print(fpioa.get_pin_func(0))                    # 获取引脚0当前功能
fpioa.help(0)                                    # 查看引脚0配置
```

2.3 PWM - 脉冲宽度调制

K230 内部包含 **2 个 PWM 硬件模块**，每个模块 3 个输出通道（共 6 个通道，编号 0-5）。通道 0/1/2 共享同一频率，通道 3/4/5 共享同一频率。占空比独立可调。

类： `machine.PWM`

方法	语法	说明
构造函数	`PWM(channel, freq, duty=50, enable=False)`	`channel`: 0-5; `freq`: 输出频率Hz; `duty`: 占空比 0-100; `enable`: 是否立即使能
freq	`PWM.freq([freq])`	获取或设置频率。不传参返回当前频率值
duty	`PWM.duty([duty])`	获取或设置占空比 0-100。不传参返回当前占空比值
enable	`PWM.enable(enable)`	使能（True）或禁用（False）输出
deinit	`PWM.deinit()`	释放 PWM 通道资源

```
from machine import FPIOA, PWM
fpioa = FPIOA()
fpioa.set_function(42, FPIOA.PWM0)
pwm0 = PWM(0, 1000, 50, enable=True)   # 通道0, 1kHz, 50%占空比
pwm0.freq(2000)                          # 改为2kHz
pwm0.duty(10)                            # 改为10%占空比
pwm0.deinit()
```

2.4 UART - 串口通信

K230 内部集成 5 个 UART：UART0(小核占用)、UART3(大核占用)，**UART1、UART2、UART4 可供用户使用。**

类： `machine.UART`

方法	语法	说明

方法	语法	说明
构造函数	`UART(id, baudrate=115200, bits=EIGHTBITS, parity=PARITY_NONE, stop=STOPBITS_ONE, timeout=0)`	`id`: `UART1`/`UART2`/`UART4`; `bits`: `FIVEBITS`~`EIGHTBITS`; `parity`: `PARITY_NONE`/`ODD`/`EVEN`; `stop`: `STOPBITS_ONE`/`TWO`; `timeout`: 读超时ms
init	`UART.init(baudrate, bits, parity, stop)`	配置 UART 参数，参数同构造函数
read	`UART.read([nbytes])`	读取数据。指定 nbytes 读最多该字节数; 否则多读。返回 bytes 对象
readline	`UART.readline()`	读取一行数据（以换行符结束），返回 bytes 对象
readinto	`UART.readinto(buf[, nbytes])`	读取字节到 buf(bytearray)。返回读取并存入的字节数
write	`UART.write(buf)`	写入数据，返回写入的字节数
deinit	`UART.deinit()`	释放 UART 资源

```
from machine import UART, FPIOA
fpioa = FPIOA()
fpioa.set_function(11, FPIOA.UART2_TXD)
fpioa.set_function(12, FPIOA.UART2_RXD)
u2 = UART(UART.UART2, baudrate=115200)
u2.write("Hello K230")
data = u2.read()
u2.deinit()
```

2.5 ADC - 模数转换

K230 内部集成 **6 通道 ADC**（编号 0-5），采样分辨率 **12 位**（输出范围 0-4095），采样速率 1 MHz。

类：`machine.ADC`

方法	语法	说明
构造函数	`ADC(channel)`	创建对象，`channel`: 0-5
read_u16	`ADC.read_u16()`	获取采样值 0-4095
read_uv	`ADC.read_uv()`	获取电压值（微伏 μV），范围 0-1800000 μV

```
from machine import ADC
adc = ADC(0)
print(adc.read_u16())    # 采样值 0-4095
print(adc.read_uv(), "uV")
```

2.6 RTC - 实时时钟

类：`machine.RTC`

方法	语法	说明
构造函数	`rtc = RTC()`	创建对象，无参数

方法	语法	说明
init	`rtc.init(year, month, day, weekday, hours, minutes, seconds, microsec)`	设置时间。`weekday`: [1-7]
datetime	`rtc.datetime()`	获取当前时间，返回 `(year, mon, day, wday, hour, min, sec, microsec)`

```
from machine import RTC
rtc = RTC()
rtc.init((2024, 2, 28, 2, 23, 59, 0, 0))
print(rtc.datetime())   # (2024, 2, 28, 2, 23, 59, 0, 0)
```

2.7 Timer - 定时器

K230 内部集成 6 个 Timer，最小定时周期 1 ms。**注意：硬件定时器 [0-5] 暂不可用，仅支持软件定时器 (index=-1)。**

类： `machine.Timer`

方法	语法	说明
构造函数	`Timer(index, mode=PERIODIC, freq=-1, period=-1, callback=None, arg=None)`	`index`: -1(软件定时器); `mode`: `ONE_SHOT`/`PERIODIC`; `freq`: 频率 Hz（优先级高于period）; `period`: 周期ms; `callback`: 回调函数必须设置且带一个参数
init	`Timer.init(mode, freq, period, callback, arg)`	初始化定时器参数，同构造函数
deinit	`Timer.deinit()`	释放定时器资源

```
from machine import Timer
tim = Timer(-1)  # 软件定时器
tim.init(period=100, mode=Timer.ONE_SHOT, callback=lambda t: print("tick"))
tim.init(period=1000, mode=Timer.PERIODIC, callback=lambda t: print("periodic"))
tim.deinit()
```

2.8 WDT - 看门狗

K230 内部集成 2 个 WDT 硬件模块。**注意：WDT0 暂不可用，仅 WDT1 可用。**

类： `machine.WDT`

方法	语法	说明
构造函数	`WDT(id=1, timeout=5)`	`id`: 0-1（默认1）; `timeout`: 超时时间秒（默认5）
feed	`WDT.feed()`	喂狗操作，防止系统复位

```
from machine import WDT
wdt = WDT(1, 3)   # 看门狗1，超时3秒
wdt.feed()          # 定期喂狗
```

3. 多媒体模块 (MPP)

3.1 Sensor - 摄像头传感器

负责图像采集与数据处理。最多支持 **3 路图像传感器**同时接入，每个视频通道可并行输出 3 路图像数据。

支持的传感器： OV5647（最高2592x1944）、GC2093（最高1920x1080）、IMX335（最高2592x1944）

类： `media.sensor.Sensor`

方法	语法	说明
构造函数	`Sensor(id, [width, height, fps])`	`id`: CSI端口 0-2（默认按硬件原理图）；`width` 默认1920; `height`默认1080; `fps`默认30
reset	`sensor.reset()`	复位传感器（**构造后必须先调用**）
set_framesize	`sensor.set_framesize(framesize=INVALID, chn=CAM_CHN_ID_0, width, height)`	设置输出尺寸。可用 `framesize` 常量或直接指定 `width`/`height`
set_pixformat	`sensor.set_pixformat(pix_format, chn=CAM_CHN_ID_0)`	设置像素格式
set_hmirror	`sensor.set_hmirror(enable)`	水平镜像 True/False
set_vflip	`sensor.set_vflip(enable)`	垂直翻转 True/False
run	`sensor.run()`	启动图像输出（**必须在 MediaManager.init() 之前**）
stop	`sensor.stop()`	停止图像输出（**必须在 MediaManager.deinit() 之前**）
snapshot	`sensor.snapshot(chn=CAM_CHN_ID_0)`	捕获一帧图像，返回 image 对象

像素格式常量：

常量	说明
`Sensor.RGB565`	16位 RGB 格式
`Sensor.RGB888`	24位 RGB 格式
`Sensor.RGBP888`	分离的 24位 RGB
`Sensor.YUV420SP`	半平面 YUV (NV12)
`Sensor.GRAYSCALE`	灰度图

通道常量： `CAM_CHN_ID_0` / `CAM_CHN_ID_1` / `CAM_CHN_ID_2`

预定义分辨率 (frame_size)： QQCIF(88x72), QCIF(176x144), QQVGA(160x120), QVGA(320x240), VGA(640x480), HD(1280x720), FHD(1920x1080), QHD(2560x1440) 等

```
from media.sensor import *
from media.display import *
from media.media import *
sensor = Sensor(id=0, width=640, height=480)
sensor.reset()
sensor.set_framesize(width=640, height=480)
sensor.set_pixformat(Sensor.RGB565)
```

```
Display.init(Display.VIRT, width=640, height=480, to_ide=True)
sensor.run()
MediaManager.init()
img = sensor.snapshot()
Display.show_image(img)
sensor.stop()
MediaManager.deinit()
Display.deinit()
```

3.2 Display - 图像显示

提供 **2 层视频图层 + 4 层 OSD 图层**支持。

导入： `from media.display import *`

Display 静态方法：

方法	语法	说明
init	`Display.init(type, width, height, osd_num=1, to_ide=False, fps, quality=90)`	初始化显示通路。需在 MediaManager.init() 之前调用
show_image	`Display.show_image(img, x=0, y=0, layer=None, alpha=255, flag=0)`	在屏幕上显示图像。可指定 layer 显示到 OSD 层
deinit	`Display.deinit()`	关闭显示通路。需在 MediaManager.deinit() 之前、sensor.stop() 之后调用
bind_layer	`Display.bind_layer(src=(mod,dev,layer), dstlayer, rect, pix_format, alpha, flag)`	将 sensor/vdec 输出直接绑定到显示层

显示类型 (type)：

类型	默认分辨率	备注
`LT9611`	1920x1080@30	HDMI 输出
`ST7701`	800x480@30	MIPI屏幕，竖屏480x800
`HX8377`	1080x1920@30	MIPI竖屏
`VIRT`	640x480@90	IDE调试，不输出外接屏

显示层： `LAYER_VIDEO1`/`LAYER_VIDEO2`(仅bind_layer) 和 `LAYER_OSD0`~`LAYER_OSD3`(show_image+bind_layer)

旋转镜像标志： `FLAG_ROTATION_0`~`FLAG_ROTATION_270`, `FLAG_MIRROR_NONE`/`HOR`/`VER`/`BOTH`

3.3 Audio - 音频模块

wave 子模块 - WAV 文件读写

导入： `import wave`

函数	语法	说明
open	`wave.open(f, mode)`	打开 WAV 文件。`mode`: 'r'/'rb'/'w'/'wb'。返回 Wave_read 或 Wave_write 对象

Wave_read 类：

方法	说明
`get_channels()`	获取声道数
`get_sampwidth()`	获取采样字节长度。返回值 1/2/3/4 对应 8/16/24/32 位
`get_framerate()`	获取采样率 8000-192000
`read_frames(nframes)`	读取 nframes 帧，返回 bytes

Wave_write 类：

方法	说明
`set_channels(nchannels)`	设置声道数
`set_sampwidth(sampwidth)`	设置采样字节长度 1/2/3/4
`set_framerate(framerate)`	设置采样率 8000-192000
`write_frames(data)`	写入 bytes 音频数据

pyaudio 子模块 - PCM 音频处理

导入： `import pyaudio`

PyAudio 类：

方法	说明
`open(**kwargs)`	打开一路音频流，返回 Stream 对象
`close(stream)`	关闭流并从 PyAudio 删除
`terminate()`	释放所有音频资源

Stream 类：

方法	说明
`__init__(manager, rate, channels, format, input=False, output=False, ...)`	构造函数。`rate`: 采样率; `channels`: 声道数; `format`: 采样字节数; `input_device_index`/`output_device_index`: [0,1]; `enable_codec`: 启用编解码; `frames_per_buffer`: 缓冲帧数; `start`: 立即启动
`read(frames)`	读取帧数，返回 bytes
`write(data)`	写入 bytes 音频数据
`volume(vol=None, channel=LEFT_RIGHT)`	设置或获取音量
`start_stream()` / `stop_stream()`	启动/停止流
`enable_audio3a(value)`	使能音频3A: `AUDIO_3A_ENABLE_ANS`(降噪)/`AGC`(自动增益)/`AEC`(回声抑制)
`audio3a_send_far_echo_frame(data, len)`	发送远端参考语音（AEC用）

```
import pyaudio, wave
wf = wave.open("test.wav", 'rb')
p = pyaudio.PyAudio()
```

```
stream = p.open(format=p.get_format_from_width(wf.get_sampwidth()),
                channels=wf.get_channels(), rate=wf.get_framerate(),
                output=True)
data = wf.read_frames(1024)
while data:
    stream.write(data)
    data = wf.read_frames(1024)
stream.stop_stream(); p.terminate()
```

3.4 MediaManager - 媒体管理器

导入： `from media.media import *`

方法	语法	说明
init	`MediaManager.init()`	初始化媒体缓冲区。在所有 buffer 配置完成后最后调用
deinit	`MediaManager.deinit()`	销毁所有 buffer
_config	`MediaManager._config(config)`	配置媒体缓冲区（内部使用）
link	`MediaManager.link(src, dst)`	连接不同模块通道，返回 linker 对象（内部使用）

调用顺序（严格遵守）：

```
1. Sensor.reset() / Display.init() 等初始化
2. sensor.run()
3. MediaManager.init()
4. ... 媒体操作 ...
5. sensor.stop()
6. MediaManager.deinit()
7. Display.deinit()
```

MediaManager.Buffer：

方法	说明
`MediaManager.Buffer.get(size)`	获取 size 字节的 buffer（init 后可用）
`buffer.__del__()`	手动释放 buffer

3.5 VDEC - 视频解码

支持 **H.264 和 H.265 解码**，最多 4 路并发，可与 VO 模块绑定直接显示。

类： `Decoder`

方法	语法	说明
构造函数	`Decoder(K_PT_H264)`	指定编码类型：`K_PT_H264` 或 `K_PT_H265`
create	`decoder.create()`	创建解码器实例
start	`decoder.Start()`	启动解码过程
decode	`decoder.decode(stream_data)`	解码一帧数据（bytes 或 StreamData）
stop	`decoder.stop()`	释放当前解码帧缓冲区

方法	语法	说明
destroy	`decoder.destroy()`	销毁解码器实例
bind_info	`vdec.bind_info(width, height, chn=0)`	获取绑定信息，用于 Display.bind_layer

StreamData 结构： `data`(码流数据) + `pts`(时间戳)

3.6 VENC - 视频编码

支持 **H.264/H.265 编码**，最多 4 路（建议使用 [0,2]，第4路固定用于 IDE 图像传输）。

类： `Encoder`

方法	语法	说明
SetOutBufs	`Encoder.SetOutBufs(chn, buf_num, width, height)`	配置输出缓冲区（**需在 MediaManager.init() 前调用**）
Create	`Encoder.Create(chn, chnAttr)`	创建编码器。chn: 0-3; chnAttr: ChnAttrStr 对象
Start	`Encoder.Start(chn)`	启动编码
SendFrame	`Encoder.SendFrame(chn, frame_info)`	发送图像数据开始编码
GetStream	`Encoder.GetStream(chn, streamData)`	获取一帧编码码流
ReleaseStream	`Encoder.ReleaseStream(chn, streamData)`	释放一帧码流缓冲区
Stop	`Encoder.Stop(chn)`	停止编码
Destroy	`Encoder.Destroy(chn)`	销毁编码器

ChnAttrStr 结构： `payload_type`(K_PT_H264/H265), `profile`(H264_BASELINE/MAIN/HIGH 或 H265_MAIN), `pic_width`, `pic_height`, `gop_len`(默认30)

StreamData 结构： `data[12]`(码流数据地址数组), `data_size[12]`, `stream_type[12]`(HEADER/I/P), `pack_cnt`

3.7 MP4 - 多媒体封装

用于生成 MP4 文件，当前仅支持 muxer 模式。

类： `Mp4Container`

方法	说明
`Create(mp4Cfg)`	创建 MP4 容器。mp4Cfg: Mp4CfgStr 对象
`Start()`	启动容器
`Process()`	将一帧音/视频数据写入 MP4 文件
`Stop()`	停止容器
`Destroy()`	销毁容器

Mp4CfgStr： `type`(MP4_CONFIG_TYPE_MUXER), `SetMuxerCfg(fileName, videoPayloadType, picWidth, picHeight, audioPayloadType, fmp4Flag=0)`

编码常量： `MP4_CODEC_ID_H264`/`MP4_CODEC_ID_H265`, `MP4_CODEC_ID_G711U`/`MP4_CODEC_ID_G711A`

3.8 Player - 播放器

支持 MP4 格式播放，同步播放音频（G.711A/U）和视频（H.264/H.265）。

类： `Player`

方法	说明
`Player(display_type, [display_to_ide])`	构造函数。display_type: `Display.VIRT`/`ST7701`/`LT9611`
`load(filename)`	加载 MP4 文件
`start()`	开始播放
`pause()`	暂停播放
`resume()`	恢复播放
`stop()`	停止播放
`set_event_callback(callback)`	设置事件回调。事件类型: `K_PLAYER_EVENT_EOF`(播放结束)/`K_PLAYER_EVENT_PROGRESS`(进度)

3.9 RTSP - 实时流媒体

模块： `multimedia`（导入 `import multimedia as mm`）

函数	语法	说明
rtspserver_create	`mm.rtsp_server().rtspserver_create()`	创建 RTSP 服务器
rtspserver_init	`rtspserver_init(port)`	初始化，监听端口（如 8554）
rtspserver_createsession	`rtspserver_createsession(name, video_type, enable_audio)`	创建会话。video_type: "h264"/"h265"
rtspserver_start	`rtspserver_start()`	启动服务器
rtspserver_getrtspurl	`rtspserver_getrtspurl()`	获取 RTSP URL
rtspserver_sendvideodata	`rtspserver_sendvideodata(session, data, size, timestamp)`	发送视频数据
rtspserver_sendaudiodata	`rtspserver_sendaudiodata(session, data, size, timestamp)`	发送音频数据
rtspserver_stop	`rtspserver_stop()`	停止服务器
rtspserver_destroysession	`rtspserver_destroysession(name)`	销毁会话
rtspserver_deinit	`rtspserver_deinit()`	反初始化
rtspserver_destroy	`rtspserver_destroy()`	销毁服务器

3.10 PM - 功耗管理

导入： `from mpp import pm`

提供 `cpu` 和 `kpu` 两个功耗域实例。

方法	语法	说明
get_freq	`pm.cpu.get_freq()` / `pm.kpu.get_freq()`	获取当前频率
list_profiles	`pm.cpu.list_profiles()`	获取支持的频率配置列表
set_profile	`pm.cpu.set_profile(index)`	设置频率配置索引

```
from mpp import pm
print(pm.cpu.get_freq())
profiles = pm.cpu.list_profiles()
pm.cpu.set_profile(1)
```

2.9 I2C - I2C通信

K230 配备 5 个硬件 I2C（I2C0-I2C4）和 5 个软件 I2C（5-9）。支持标准 100 kb/s、快速 400 kb/s 和高速 3.4 Mb/s。

类：`machine.I2C`（主设备）

方法	语法	说明
构造函数	`I2C(id, freq=100000, timeout=1000, [scl, sda])`	`id`: 0-4 硬件I2C, 5-9 软件I2C（需传scl/sda引脚）
scan	`i2c.scan()`	扫描总线从机，返回地址列表
readfrom	`i2c.readfrom(addr, len, True)`	从从机读取指定长度数据，返回 bytes
readfrom_into	`i2c.readfrom_into(addr, buf, True)`	读取数据到 bytearray 缓冲区
writeto	`i2c.writeto(addr, buf, True)`	向从机发送数据，返回成功发送字节数
readfrom_mem	`i2c.readfrom_mem(addr, memaddr, nbytes, mem_size=8)`	从存储设备读取。`mem_size`: 地址宽度 8/16位
readfrom_mem_into	`i2c.readfrom_mem_into(addr, memaddr, buf, mem_size=8)`	读存储器到缓冲区
writeto_mem	`i2c.writeto_mem(addr, memaddr, buf, mem_size=8)`	向存储设备写入
deinit	`i2c.deinit()`	释放 I2C 资源

类：`machine.I2C_Slave`（从设备模式，需定制固件启用）

方法	语法	说明
list	`I2C_Slave.list()`	获取可用从设备 ID 列表
构造函数	`I2C_Slave(id, addr=0x10, mem_size=20)`	创建从设备对象
readfrom_mem	`i2c.readfrom_mem(addr, n)`	从映射内存读 n 字节
writeto_mem	`i2c.writeto_mem(addr, data)`	写 bytearray 到映射内存

```
from machine import I2C
i2c = I2C(0, freq=100000)
addrs = i2c.scan()                      # 扫描从机
```

```
data = i2c.readfrom(0x50, 4, True)        # 从地址0x50读4字节
i2c.writeto(0x50, b'hello', True)         # 向地址0x50写数据
i2c.deinit()
```

2.10 SPI - SPI通信

K230 内部集成 **3 个 SPI 硬件模块**（SPI0-SPI2）。时钟速率公式: `Fsclk_out = Fssi_clk / BAUDR`。

类: `machine.SPI`

方法	语法	说明
构造函数	`SPI(id, baudrate=20, polarity=0, phase=0, bits=8)`	`id`: 0-2; `baudrate`: 时钟速率; `polarity`: 时钟极性0/1; `phase`: 时钟相位0/1; `bits`: 数据位宽
read	`spi.read(nbytes)`	读取指定字节数，返回 bytes
readinto	`spi.readinto(buf)`	读取数据到 bytearray 缓冲区
write	`spi.write(buf)`	发送 bytearray 数据
write_readinto	`spi.write_readinto(write_buf, read_buf)`	发送的同时接收数据到 read_buf
deinit	`spi.deinit()`	注销 SPI 模块

```
from machine import SPI
spi = SPI(1, baudrate=5000000, polarity=0, phase=0, bits=8)
spi.write(b'\xAA\xBB')
read_buf = bytearray(4)
spi.write_readinto(b'\x01\x02', read_buf)
spi.deinit()
```

2.11 TOUCH - 触摸屏

基于 RTT 触摸框架，支持单点和多点电容触摸屏及电阻触摸屏。

类: `machine.TOUCH`

方法	语法	说明
构造函数	`TOUCH(index, type=CST328, rotate=-1, ...)`	`index=0`: 系统自带触摸; `index=1`: CanMV 专有驱动（需额外参数 range_x/y, i2c, slave_addr, rst, int）
read	`TOUCH.read([count])`	获取触摸点数据，返回 `(touch_info, ...)` 元组
deinit	`TOUCH.deinit()`	释放触摸资源

TOUCH_INFO 属性:

- `event`: `EVENT_NONE` / `EVENT_UP` / `EVENT_DOWN` / `EVENT_MOVE`
- `track_id`: 触点 ID; `width`: 触点宽度; `x`, `y`: 坐标; `timestamp`: 时间戳

常量: 触摸类型 `TYPE_CST328`/`TYPE_CST226SE`/`TYPE_GT911`; 旋转 `ROTATE_0`~`ROTATE_270`

```
```

```
from machine import TOUCH
tp = TOUCH(0)
p = tp.read()
for pt in p:
    print(f"x={pt.x}, y={pt.y}, event={pt.event}")
```

2.12 LED - LED控制

控制开发板上的三原色 LED 灯珠，亮度范围 0-255。

类： `machine.LED`

方法	语法	说明
构造函数	`LED(id)`	`id`: 1或"LED_RED"/2或"LED_GREEN"/3或"LED_BLUE"
value	`led.value(val)`	设置亮度 0-255
on	`led.on()`	最大亮度点亮
off	`led.off()`	熄灭 LED

```
from machine import LED
import time
red = LED("LED_RED")
green = LED(2)
red.on(); time.sleep_ms(500); red.off()
green.value(128)   # 半亮
```

2.13 Machine 基模块

`machine` **模块自带方法：**

方法	语法	说明
reset	`machine.reset()`	立即复位 SoC
mem_copy	`machine.mem_copy(dst, src, size)`	将 size 字节从 src 地址复制到 dst 地址
temperature	`machine.temperature()`	获取芯片内部温度（浮点数）
chipid	`machine.chipid()`	获取芯片 ID（bytearray，长度32）

2.14 FFT - 快速傅里叶变换

类： `machine.FFT`

方法	语法	说明
构造函数	`FFT(data, points, shift=0)`	`data`: bytearray 时域数据; `points`: 64/128/256/512/1024/2048/4096; `shift`: 偏移量
run	`fft.run()`	执行 FFT，返回 `[(实部,虚部),...]` 含 points 个元组
freq	`FFT.freq(points, sample_rate)`	静态方法，计算频率值列表

方法	语法	说明
amplitude	`FFT.amplitude(FFT_res)`	静态方法，从 run() 结果计算幅值列表

```
from machine import FFT
import array
data = array.array('i', [1,2,3,4,5,6,7,8])
fft = FFT(data, 64, 0)
res = fft.run()          # [(real, imag),...]
amps = FFT.amplitude(res) # 幅值列表
```

2.15 SPI_LCD - SPI屏幕驱动

类： `machine.SPI_LCD`

方法	语法	说明
构造函数	`SPI_LCD(spi, dc, cs=None, rst=None, bl=None, type=ST7789)`	`spi`: SPI对象; `dc`: 数据/命令Pin; `cs`: 片选Pin; `rst`: 复位Pin; `bl`: 背光Pin; `type`: `SPI_LCD.ST7789`
configure	`lcd.configure(width, height, hmirror=False, vflip=False, bgr=False)`	配置屏幕参数。需在 init 前调用
init	`lcd.init(custom_command=False)`	初始化屏幕，返回 Image 对象（绘制缓冲）
show	`lcd.show(img=None, x=0, y=0)`	显示图像。不传 img 则显示内部缓冲
show_image	`lcd.show_image(img=None, x=0, y=0)`	同 show
fill	`lcd.fill(color)`	用指定颜色填充（操作缓冲，需 show 后显示）
pixel	`lcd.pixel(x, y, color)`	在指定位置画像素
set	`lcd.set(x, y, color)`	同 pixel
get	`lcd.get(x, y)`	获取指定位置像素值
light	`lcd.light(value)`	设置背光亮度 0-100
command	`lcd.command(cmd, data)`	发送原始命令（用于自定义初始化）
deinit	`lcd.deinit()`	关闭并释放屏幕
width	`lcd.width()`	获取屏幕宽度
height	`lcd.height()`	获取屏幕高度
hmirror	`lcd.hmirror([val])`	获取/设置水平镜像
vflip	`lcd.vflip([val])`	获取/设置垂直翻转
bgr	`lcd.bgr([val])`	获取/设置 BGR 颜色格式
get_direction	`lcd.get_direction()`	获取方向寄存器值

```
from machine import FPIOA, Pin, SPI, SPI_LCD
fpioa = FPIOA()
fpioa.set_function(19, FPIOA.GPIO19)
fpioa.set_function(20, FPIOA.GPIO20)
```

```
fpioa.set_function(15, fpioa.QSPI0_CLK)
fpioa.set_function(16, fpioa.QSPI0_D0)
pin_cs = Pin(19, Pin.OUT); pin_dc = Pin(20, Pin.OUT)
spi1 = SPI(1, baudrate=50000000, polarity=1, phase=1, bits=8)
lcd = SPI_LCD(spi1, pin_dc, pin_cs)
lcd.configure(320, 240, hmirror=False, vflip=True, bgr=False)
img = lcd.init()
img.clear()
img.draw_string_advanced(0, 0, 32, "Hello K230", color=(255, 0, 0))
lcd.show()
```

> 本文档系统总结了庐山派 K230-CanMV 开发板的各模块函数和类的功能、参数及用法。所有内容均展开为内联说明，无需联网查阅外部 URL，可作为离线字典随用随查。

目录

- 1. 概述
- 2. 基础外设模块 (Machine)
- 3. 多媒体模块 (MPP)
- 4. 图像处理模块 (OpenMV兼容)
- 5. 网络模块
- 6. AI Demo 模块
- 7. 标准库与扩展库
- 8. 完整例程速查

---

4. 图像处理模块 (OpenMV兼容)

此模块移植自 OpenMV，提供丰富的图像处理功能。调用方式：`import image`

支持的图像格式： `BINARY`, `GRAYSCALE`, `RGB565`, `BAYER`, `YUV422`, `JPEG`, `PNG`, `ARGB8888`, `RGB888`, `RGBP888`, `YUV420`

内存分配区域： `ALLOC_MPGC`(MicroPython GC), `ALLOC_HEAP`(系统堆), `ALLOC_MMZ`(多媒体内存), `ALLOC_VB`(视频缓冲), `ALLOC_REF`(引用外部内存)

4.1 Image 类 - 构造函数

```
# 从文件创建
image.Image(path, alloc=ALLOC_MMZ, cache=True, phyaddr=0, virtaddr=0, poolid=0, data=None)
# 创建空白图像
image.Image(w, h, format, alloc=ALLOC_MMZ, cache=True, phyaddr=0, virtaddr=0, poolid=0,
data=None)
```

参数： `w`/`h`: 宽高; `format`: 图像格式; `alloc`: 内存分配方式; `cache`: 启用缓存; `phyaddr`/`virtaddr`/`poolid`: VB区域参数; `data`: 引用外部数据

4.2 基础属性/方法

方法	说明
`image.width()`	返回宽度（像素）
`image.height()`	返回高度（像素）

方法	说明
`image.format()`	返回格式
`image.size()`	返回字节大小
`image.phyaddr()`	物理内存地址
`image.virtaddr()`	虚拟内存地址
`image.poolid()`	VB 池 ID
`image.get_pixel(x, y)`	获取像素值。灰度图返回灰度值; RGB565返回(r,g,b)
`image.set_pixel(x, y, pixel)`	设置像素值

4.3 图像转换

方法	说明
`image.to_rgb888(x_scale, y_scale, roi, ...)`	转为RGB888
`image.to_grayscale([copy=False])`	转为灰度图
`image.to_rgb565([copy=False])`	转为RGB565
`image.to_rainbow([copy=False])`	转为彩虹色
`image.copy([roi])`	创建副本
`image.copy_from(src_img)`	从src复制到当前
`image.copy_to(dst_img)`	复制到dst
`image.to_numpy_ref()`	转NumPy数组（共享内存）
`image.compress([quality=50])`	JPEG压缩
`image.compress_for_ide([quality=50])`	IDE显示压缩
`image.compressed([quality=50])`	返回压缩后图像
`image.save(path[, roi[, quality=50]])`	保存到文件
`image.clear()`	快速清零所有像素

4.4 绘图方法

方法	语法	说明
draw_line	`image.draw_line(x0,y0,x1,y1[, color[, thickness=1]])`	绘制直线。color: RGB888元组，默认白色
draw_rectangle	`image.draw_rectangle(x,y,w,h[, color[, thickness=1[, fill=False]]])`	绘制矩形
draw_circle	`image.draw_circle(x,y,radius[, color[, thickness=1[, fill=False]]])`	绘制圆
draw_ellipse	`image.draw_ellipse(cx,cy,rx,ry,rot[, color[, thickness=1[, fill=False]]])`	绘制椭圆

方法	语法	说明
draw_cross	`image.draw_cross(x,y[, color[, size=5[, thickness=1]]])`	绘制十字标记
draw_arrow	`image.draw_arrow(x0,y0,x1,y1[, color[, thickness=1]])`	绘制箭头
draw_string	`image.draw_string(x,y,text[, color[, scale=1[, x_spacing=0[, y_spacing=0[, mono_space=True]]]]])`	绘制8x10英文文本
draw_string_advanced	`image.draw_string_advanced(x, y, char_size, str[, color[, font]])`	增强文本（支持中文和自定义字体）
draw_image	`image.draw_image(img,x,y[, x_scale=1.0[, y_scale=1.0[, mask[, alpha=256]]]])`	绘制另一图像。alpha: 0-256
draw_keypoints	`image.draw_keypoints(keypoints[, color[, size=10[, thickness=1[, fill=False]]]])`	绘制特征点

4.5 图像处理算法

方法	说明
`binary(thresholds[, invert=False[, zero=False[, mask]]])`	二值化。thresholds: `[(lo,hi),...]`
`invert()`	反转二值图像
`b_and(img[, mask])` / `b_nand(img[, mask])`	按位与/与非
`b_or(img[, mask])` / `b_nor(img[, mask])`	按位或/或非
`b_xor(img[, mask])`	按位异或
`mean_pool(x_div, y_div)`	均值池化
`mean_pooled(x_div, y_div)`	均值池化（返回副本）
`midpoint_pool(x_div, y_div[, bias=0.5])`	中点池化
`midpoint_pooled(x_div, y_div[, bias=0.5])`	中点池化（返回副本）
`flood_fill(x, y[, seed_threshold=0.05[, floating_threshold=0.05[, color[, invert[, clear_background[, mask]]]]]])`	泛洪填充
`histeq([mask])`	直方图均衡化
`gamma_corr(gamma)`	Gamma校正
`negate()`	负片效果
`find_blobs(thresholds[, ...])`	查找色块
`find_lines(thresholds[, ...])`	查找直线
`find_circles(thresholds[, ...])`	查找圆形
`find_rects([threshold])`	查找矩形
`find_apriltags([...])`	查找AprilTag
`find_qrcodes([...])`	查找二维码
`find_datamatrices([...])`	查找DataMatrix码

方法	说明
`find_barcodes([...])`	查找条形码
`find_keypoints([...])`	查找特征点
`find_lbp(roi)`	查找LBP特征
`find_edges(edge_type[, threshold])`	边缘检测
`find_hog([...])`	HOG特征检测

```
import image
img = image.Image(800, 480, image.RGB565)
img.clear()
img.draw_string_advanced(0, 0, 32, "Hello K230!", color=(255, 0, 0))
img.draw_rectangle(50, 50, 100, 80, color=(0, 255, 0), thickness=2, fill=False)
```

5. 网络模块

5.1 network - 网络配置

导入： `import network`

LAN 类 - 有线以太网

方法	语法	说明
构造函数	`LAN()`	创建有线以太网对象
active	`LAN.active([state])`	激活/停用（True/False）或查询状态
isconnected	`LAN.isconnected()`	返回 True/False
ifconfig	`LAN.ifconfig([(ip, subnet, gateway, dns)])`	获取/设置 IP 参数
config	`LAN.config(param)`	设置 MAC 地址: `lan.config(mac="42:EA:D0:C2:0D:83")`

WLAN 类 - WiFi 网络

接口类型： `network.STA_IF`(客户端) 和 `network.AP_IF`(热点)

方法	说明
`WLAN(interface_id)`	构造函数。`interface_id`: `network.STA_IF` 或 `network.AP_IF`
`active([is_active])`	激活/停用或查询状态
`connect(ssid, password, bssid)`	连接WiFi。（仅STA模式）
`disconnect()`	断开当前连接
`scan()`	扫描可用WiFi。返回列表[(ssid, bssid, channel, RSSI, security, hidden)]
`status([param])`	获取连接状态详情（BSSID/频率/SSID/IP等）

方法	说明
`isconnected()`	STA模式: 已连接且有IP返回True; AP模式: 有设备连接返回True
`ifconfig([(ip, subnet, gateway, dns)])`	获取/设置IP参数
`config(param)`	配置参数。支持: `mac`, `ssid`, `channel`, `hidden`, `password`

```
import network
# STA模式
sta = network.WLAN(network.STA_IF)
sta.active(True)
sta.connect('my-ssid', 'my-password')
while not sta.isconnected(): pass
print("IP:", sta.ifconfig()[0])

# AP模式
ap = network.WLAN(network.AP_IF)
ap.config(ssid='K230_AP', channel=11, key='12345678')
ap.active(True)
```

5.2 socket - 网络通信

导入： `import socket`

构造函数： `socket.socket(af=AF_INET, type=SOCK_STREAM, proto=IPPROTO_TCP)`
TCP: `socket(AF_INET, SOCK_STREAM)` / UDP: `socket(AF_INET, SOCK_DGRAM)`

方法	说明
`socket.close()`	关闭套接字
`socket.bind(address)`	绑定到指定 IP:端口
`socket.listen([backlog])`	开始监听连接
`socket.accept()`	接受连接，返回 `(conn, address)`
`socket.connect(address)`	连接到服务端
`socket.send(bytes)`	发送数据，返回发送字节数
`socket.sendall(bytes)`	发送全部数据
`socket.recv(bufsize)`	接收数据，返回 bytes
`socket.sendto(bytes, address)`	向指定地址发送（UDP）
`socket.recvfrom(bufsize)`	接收数据和源地址（UDP），返回 `(bytes, address)`
`socket.setsockopt(level, optname, value)`	设置选项
`socket.settimeout(value)`	设置超时秒数
`socket.setblocking(flag)`	设置阻塞模式
`socket.read([size])`	读取数据
`socket.readline()`	读取一行

方法	说明
`socket.write(buf)`	写入数据

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM, 0)
ai = socket.getaddrinfo("10.10.1.94", 60000)
s.connect(ai[0][-1])
s.write("Hello from K230")
data = s.read(1024)
s.close()
```

6. AI Demo 模块

AI Demo 模块提供了从摄像头获取图像、AI 推理到显示结果的完整流程框架。

6.1 PipeLine - AI流程管道

封装了单摄双通道默认配置：一路 Camera 图像直接送 Display 显示，另一路通过 `get_frame` 获取供 AI 程序使用。

导入： `from libs.PipeLine import PipeLine, ScopedTiming`

类： `PipeLine`

方法	语法	说明
构造函数	`PipeLine(rgb888p_size=[1920,1080], display_size=[1920,1080], display_mode='hdmi', debug_mode=0)`	`rgb888p_size`: AI输入分辨率; `display_size`: 显示分辨率; `display_mode`: 'hdmi'或'lcd'; `debug_mode`: 0计时/1不计时
create	`pl.create(sensor=None, hmirror=None, vflip=None, fps=60)`	初始化Sensor/Display/OSD。可选传入Sensor实例和镜像翻转设置
get_frame	`pl.get_frame()`	获取一帧图像，返回 ulab.numpy.ndarray（CHW格式，RGBP888）
show_image	`pl.show_image()`	将 pl.osd_img 上绘制的AI结果叠加到Display显示
destroy	`pl.destroy()`	反初始化管道，释放资源

pl.osd_img： create 后初始化的 ARGB8888 空白图像，用于绘制 AI 结果叠加层。

工具类 `ScopedTiming`： 上下文管理器，用于测量代码块执行时间。

```
with ScopedTiming("label", debug_mode):
    # 计时代码
    pass
```

6.2 Ai2d - AI预处理

封装了 nncase_runtime.ai2d 支持的预处理方法（crop/shift/pad/resize/affine）。

导入： `from libs.AI2D import Ai2d`

规则注意： (1) Affine和Resize互斥; (2) Shift仅支持Raw16输入; (3) Pad value按通道配置; (4) 多操作执行顺序: Crop→Shift→Resize/Affine→Pad

类： `Ai2d`

方法	语法	说明
构造函数	`Ai2d(debug_mode=0)`	`debug_mode`: 0计时/1不计时
set_ai2d_dtype	`ai2d.set_ai2d_dtype(input_format, output_format, input_type, output_type)`	设置输入输出格式和类型。`format`: `NCHW_FMT`/`RGB_packed`; `type`: `np.uint8`/`np.float`
crop	`ai2d.crop(start_x, start_y, width, height)`	配置裁剪
shift	`ai2d.shift(shift_val)`	配置右移比特数
pad	`ai2d.pad(paddings, pad_mode=0, pad_val)`	配置padding。paddings: 8元素list [N,C,H,W各2]; pad_val: [R,G,B]填充值
resize	`ai2d.resize(interp_method, interp_mode)`	配置缩放。method: `tf_nearest`/`tf_bilinear`/`cv2_nearest`/`cv2_bilinear`; mode: `none`/`align_corner`/`half_pixel`
affine	`ai2d.affine(interp_method, cord_round=0, bound_ind=0, bound_val=127, bound_smooth=1, M)`	配置仿射变换。M: 2x3矩阵展平list
build	`ai2d.build(input_shape, output_shape)`	构建预处理器
run	`ai2d.run(input_np)`	执行预处理，返回 nncase_runtime.tensor

```
from libs.AI2D import Ai2d
import nncase_runtime as nn
import ulab.numpy as np
ai2d = Ai2d(debug_mode=0)
ai2d.set_ai2d_dtype(nn.ai2d_format.RGB_packed, nn.ai2d_format.NCHW_FMT, np.uint8,
np.uint8)
ai2d.resize(nn.interp_method.tf_bilinear, nn.interp_mode.half_pixel)
ai2d.build([1,3,720,1280], [1,3,224,224])
output_tensor = ai2d.run(img)
```

6.3 AIBase - AI推理基类

封装了单个模型的完整推理过程。开发 AI 应用时**继承此类**并重写 `postprocess` 和 `draw_result` 方法。

导入： `from libs.AIBase import AIBase`

类： `AIBase`

方法	语法	说明
构造函数	`AIBase(kmodel_path, model_input_size, rgb888p_size, debug_mode=0)`	`kmodel_path`: 模型路径; `model_input_size`: [w,h]; `rgb888p_size`: 输入图像分辨率[w,h]

方法	语法	说明
get_kmodel_inputs_num	`aibase.get_kmodel_inputs_num()`	获取模型输入数目
get_kmodel_outputs_num	`aibase.get_kmodel_outputs_num()`	获取模型输出数目
preprocess	`aibase.preprocess(input_np)`	AI2D预处理，返回 input_tensors（可重写）
inference	`aibase.inference()`	执行推理，返回 outputs 列表（ulab.numpy.ndarray 格式）
postprocess	`aibase.postprocess(results)`	**需在子类重写** - 后处理
run	`aibase.run(input_np)`	完整流程: 预处理→推理→后处理，返回后处理输出
deinit	`aibase.deinit()`	销毁 KPU 实例，释放内存

子类开发模板:

```
from libs.AIBase import AIBase
from libs.AI2D import Ai2d
import nncase_runtime as nn
import ulab.numpy as np

class MyAIApp(AIBase):
    def __init__(self, kmodel_path, ...):
        super().__init__(kmodel_path, model_input_size, rgb888p_size, debug_mode)
        self.ai2d = Ai2d(debug_mode)
        self.ai2d.set_ai2d_dtype(...)

    def config_preprocess(self, input_image_size=None):
        self.ai2d.pad(...)
        self.ai2d.resize(...)
        self.ai2d.build(in_shape, out_shape)

    def postprocess(self, results):
        # 自定义后处理
        return post_ret

    def draw_result(self, pl, dets):
        pl.osd_img.clear()
        # 在 pl.osd_img 上绘制检测结果
```

6.4 YOLO - 目标检测

支持 YOLOv5/YOLOv8/YOLO11 训练并转换的模型，支持分类/检测/分割三类任务。

导入: `from libs.YOLO import YOLOv5`

类: `YOLOv5`

方法	语法	说明

方法	语法	说明
构造函数	`YOLOv5(task_type="detect", mode="image", kmodel_path, labels, rgb888p_size, model_input_size, display_size, conf_thresh=0.5, nms_thresh=0.25, mask_thresh, max_boxes_num=50, debug_mode=0)`	`task_type`: 'classify'/'detect'/'segment'; `mode`: 'image'/'video'; `labels`: 类别标签 list; `conf_thresh`: 置信度阈值0-1; `nms_thresh`: NMS阈值
config_preprocess	`yolo.config_preprocess()`	配置预处理（内部调用AIBase方法）
run	`yolo.run(img)`	推理一帧。分类返回(类别索引,分数); 检测返回检测框列表; 分割返回mask和检测框
draw_result	`yolo.draw_result(res, img_ori)`	绘制推理结果到 img_ori

6.5 nncase_runtime - 神经网络运行时

导入： `import nncase_runtime as nn`

kpu 模块

方法	语法	说明
load_kmodel	`kpu.load_kmodel(path)` 或 `load_kmodel(read_bin)`	加载 .kmodel 模型
set_input_tensor	`kpu.set_input_tensor(index, runtime_tensor)`	设置输入
get_input_tensor	`kpu.get_input_tensor(index)`	获取输入 tensor
set_output_tensor	`kpu.set_output_tensor(index, runtime_tensor)`	设置输出
get_output_tensor	`kpu.get_output_tensor(index)`	获取输出 tensor
run	`kpu.run()`	启动推理
inputs_size	`kpu.inputs_size()`	输入个数
outputs_size	`kpu.outputs_size()`	输出个数
get_input_desc	`kpu.get_input_desc(index)`	输入描述 {dtype, start, size}
get_output_desc	`kpu.get_output_desc(index)`	输出描述

ai2d 模块

方法	说明
`ai2d.build(input_shape, output_shape)`	构建 ai2d_builder
`ai2d_builder.run(input_tensor, output_tensor)`	执行 AI2D
`set_dtype(src_format, dst_format, src_type, dst_type)`	设置数据类型
`set_crop_param(crop_flag, start_x, start_y, w, h)`	裁剪参数
`set_resize_param(flag, method, mode)`	缩放参数
`set_shift_param(flag, shift_val)`	移位参数

方法	说明
`set_pad_param(flag, paddings, pad_mode, pad_val)`	填充参数
`set_affine_param(flag, method, cord_round, bound_ind, bound_val, bound_smooth, M)`	仿射变换

工具函数

函数	说明
`nn.from_numpy(ulab.ndarray)`	从 numpy 创建 runtime_tensor
`tensor.to_numpy()`	runtime_tensor 转 ulab.ndarray

```
import nncase_runtime as nn
import ulab.numpy as np

kpu = nn.kpu()
kpu.load_kmodel("/sdcard/examples/kmodel/model.kmodel")
input_data = np.ones((1,3,224,224), dtype=np.uint8)
input_tensor = nn.from_numpy(input_data)
kpu.set_input_tensor(0, input_tensor)
kpu.run()
for i in range(kpu.outputs_size()):
    output = kpu.get_output_tensor(i).to_numpy()
    print(output.shape)
```

7. 标准库与扩展库

7.1 utime - 时间相关

导入： `import utime`

函数	说明
`utime.ntp_sync()`	从互联网同步时间，返回 True/False
`utime.localtime([secs])`	返回8元组 (年,月,日,时,分,秒,工作日0-6,yearday)
`utime.mktime(tuple)`	localtime的逆函数，返回Unix时间戳
`utime.sleep(seconds)`	延迟指定秒数（支持浮点）
`utime.sleep_ms(ms)`	延迟指定毫秒数
`utime.sleep_us(us)`	延迟指定微秒数
`utime.ticks_ms()`	返回毫秒计数器（会回绕）
`utime.ticks_us()`	返回微秒计数器
`utime.ticks_cpu()`	返回CPU时钟计数器
`utime.ticks_add(ticks, delta)`	计算新ticks值（支持正负增量）
`utime.ticks_diff(ticks1, ticks2)`	计算两个ticks差值（处理回绕）
`utime.time()`	Unix时间戳（需RTC）或上电秒数

函数	说明
`utime.ticks()`	等同于 ticks_ms()
`utime.clock()`	返回 clock 对象

clock 类：

方法	说明
`clock.tick()`	记录当前时间（用于FPS计算）
`clock.fps()`	根据上次 tick() 后的时间计算FPS
`clock.reset()`	重置所有计时标记
`clock.avg()`	平均每帧耗时

7.2 uos - 操作系统接口

导入： `import uos`

函数	说明
`uos.uname()`	返回系统信息元组 (sysname, nodename, release, version, machine)
`uos.urandom(n)`	生成 n 个随机字节
`uos.cpu_usage()`	返回 CPU 使用率 0-100
`uos.chdir(path)`	更改当前目录
`uos.getcwd()`	获取当前目录
`uos.listdir([dir])`	列出目录条目
`uos.ilistdir([dir])`	返回迭代器，每项 (name, type, inode[, size])
`uos.mkdir(path)`	创建目录
`uos.remove(path)`	删除文件
`uos.rmdir(path)`	删除目录
`uos.rename(old, new)`	重命名文件/目录
`uos.stat(path)`	获取文件状态
`uos.statvfs(path)`	获取文件系统状态。有效字段: f_bsize, f_blocks, f_bfree
`uos.sync()`	同步文件系统
`uos.dupterm(stream, index=0)`	复制/切换终端到 stream 对象。传 None 取消

7.3 ucryptolib - 加密模块

导入： `import ucryptolib`

AES-GCM（硬件加速）： `ucryptolib.aes((key, mode, IV, AAD))`
key: 256位; mode: 0(AES-GCM); IV: 12字节; AAD: 任意长度

方法	说明

方法	说明
`cipher.encrypt(plaintext)`	加密
`cipher.decrypt(ciphertext)`	解密

SM4（国密，硬件加速）： `ucryptolib.sm4((key, mode, IV))`
key: 128位; mode: 0(ECB)/1(CBC); IV: CBC模式需16字节

```
import ucryptolib
cipher = ucryptolib.aes((key, mode=0, IV=iv, AAD=aad))
encrypted = cipher.encrypt(plaintext)
decrypted = cipher.decrypt(encrypted)
```

7.4 uhashlib - 哈希模块

导入： `import uhashlib`

SHA256（硬件加速）： `uhashlib.sha256([data])`

方法	说明
`obj.update(data)`	追加数据进行哈希
`obj.digest()`	返回哈希值 bytes。**只能调用一次**

还支持 `uhashlib.md5([data])` 和 `uhashlib.sha1([data])`。

```
import uhashlib, binascii
obj = uhashlib.sha256()
obj.update(b'hello')
obj.update(b'world')
digest = obj.digest()
print(binascii.hexlify(digest))
```

7.5 uctypes - 结构化数据访问

导入： `import uctypes`

类/函数	说明
`uctypes.struct(addr, descriptor, layout_type=NATIVE)`	在内存地址上创建结构对象
`uctypes.sizeof(struct, layout_type=NATIVE)`	返回结构字节大小
`uctypes.addressof(obj)`	返回对象内存地址
`uctypes.bytes_at(addr, size)`	获取不可变 bytes
`uctypes.bytearray_at(addr, size)`	获取可变 bytearray
`uctypes.string_at(addr, size=-1)`	获取字符串

字段定义： `"field_name": offset | uctypes.UINT32` (标量); `"sub": (offset, {...})` (嵌套); `"arr": (offset | ARRAY, size | UINT8)` (数组); `"ptr": (offset | PTR, TYPE)` (指针); `"bitf": offset | BFUINT16 | lsbit << BF_POS | bitsize << BF_LEN` (位域)

常量： LITTLE_ENDIAN, BIG_ENDIAN, NATIVE; 整数类型 UINT8/INT8~UINT64/INT64; 浮点 FLOAT32/FLOAT64; VOID, ARRAY, PTR, BF_POS, BF_LEN, BFUINT8~BFUINT32

7.6 lvgl - 图形界面库

LVGL (Light and Versatile Graphics Library) 与 MicroPython 绑定的图形界面开发库。API 参考 LVGL 官方文档。

---

8. 完整例程速查

8.1 GPIO LED 闪烁

```
from machine import Pin
import time
led = Pin(2, Pin.OUT)
while True:
    led.on(); time.sleep_ms(500)
    led.off(); time.sleep_ms(500)
```

8.2 PWM 控制舵机

```
from machine import FPIOA, PWM
import time
fpioa = FPIOA()
fpioa.set_function(42, FPIOA.PWM0)
servo = PWM(0, 50, 7.5, enable=True)  # 50Hz, 7.5%占空比 (90°)
time.sleep(2)
servo.duty(2.5)    # 0°
time.sleep(2)
servo.duty(12.5)   # 180°
servo.deinit()
```

8.3 UART 回显

```
from machine import UART, FPIOA
fpioa = FPIOA()
fpioa.set_function(11, FPIOA.UART2_TXD)
fpioa.set_function(12, FPIOA.UART2_RXD)
u = UART(UART.UART2, 115200)
while True:
    if u.any():
        u.write(u.read())
```

8.4 摄像头采集 + 显示

```
from media.sensor import *
from media.display import *
from media.media import *
import os, time

sensor = Sensor(id=0, width=640, height=480)
```

```
sensor.reset()
sensor.set_framesize(width=640, height=480)
sensor.set_pixformat(Sensor.RGB565)
Display.init(Display.VIRT, width=640, height=480, to_ide=True)
sensor.run()
MediaManager.init()

try:
    while True:
        os.exitpoint()
        img = sensor.snapshot()
        Display.show_image(img)
except KeyboardInterrupt:
    pass
finally:
    sensor.stop()
    MediaManager.deinit()
    Display.deinit()
```

8.5 AI 人脸检测 (使用 AIBase 框架)

```
from libs.PipeLine import PipeLine, ScopedTiming
from libs.AIBase import AIBase
from libs.AI2D import Ai2d
import nncase_runtime as nn
import ulab.numpy as np
import aidemo, image, gc, os

class FaceDetApp(AIBase):
    def __init__(self, kmodel_path, model_input_size, anchors, confidence_threshold=0.5,
nms_threshold=0.2, rgb888p_size=[224,224], display_size=[1920,1080], debug_mode=0):
        super().__init__(kmodel_path, model_input_size, rgb888p_size, debug_mode)
        self.confidence_threshold = confidence_threshold
        self.nms_threshold = nms_threshold
        self.anchors = anchors
        self.display_size = display_size
        self.ai2d = Ai2d(debug_mode)
        self.ai2d.set_ai2d_dtype(nn.ai2d_format.NCHW_FMT, nn.ai2d_format.NCHW_FMT,
np.uint8, np.uint8)

    def config_preprocess(self, input_image_size=None):
        sz = input_image_size or self.rgb888p_size
        top, bottom, left, right = self.get_padding_param()
        self.ai2d.pad([0,0,0,0,top,bottom,left,right], 0, [104,117,123])
        self.ai2d.resize(nn.interp_method.tf_bilinear, nn.interp_mode.half_pixel)
        self.ai2d.build([1,3,sz[1],sz[0]],
[1,3,self.model_input_size[1],self.model_input_size[0]])

    def postprocess(self, results):
        return aidemo.face_det_post_process(self.confidence_threshold, self.nms_threshold,
self.model_input_size[1], self.anchors, self.rgb888p_size, results)

    def draw_result(self, pl, dets):
        pl.osd_img.clear()
        if dets:
            for det in dets:
                x, y, w, h = map(lambda v: int(round(v, 0)), det[:4])
```

```
                x = x * self.display_size[0] // self.rgb888p_size[0]
                y = y * self.display_size[1] // self.rgb888p_size[1]
                w = w * self.display_size[0] // self.rgb888p_size[0]
                h = h * self.display_size[1] // self.rgb888p_size[1]
                pl.osd_img.draw_rectangle(x, y, w, h, color=(255,0,0,255), thickness=2)

pl = PipeLine(rgb888p_size=[1280,720], display_size=[1920,1080], display_mode='hdmi')
pl.create()
app = FaceDetApp("/data/kmodel/face_detect.kmodel", [320,320], anchors=[...],
rgb888p_size=[1280,720])
app.config_preprocess()
try:
    while True:
        os.exitpoint()
        img = pl.get_frame()
        res = app.run(img)
        app.draw_result(pl, res)
        pl.show_image()
        gc.collect()
except Exception as e:
    print(e)
finally:
    app.deinit()
    pl.destroy()
```

8.6 WiFi 连接 + HTTP 请求

```
import network, socket

sta = network.WLAN(network.STA_IF)
sta.active(True)
sta.connect('SSID', 'password')
while not sta.isconnected():
    pass

addr = socket.getaddrinfo('example.com', 80)[0][-1]
s = socket.socket()
s.connect(addr)
s.write(b'GET / HTTP/1.1\r\nHost: example.com\r\n\r\n')
print(s.read(1024))
s.close()
```

8.7 RTSP 推流

```
from media.sensor import *
from media.vencoder import *
from media.media import *
import multimedia as mm
import _thread, time

# 初始化sensor和vencoder（与VENC例程类似）
# 创建RTSP服务器
rtspserver = mm.rtsp_server()
rtspserver.rtspserver_create()
rtspserver.rtspserver_init(8554)
```

```
rtspserver.rtspserver_createsession("live", "h264", False)
rtspserver.rtspserver_start()
# 循环发送编码数据
while True:
    # encoder.GetStream + rtspserver_sendvideodata
    pass
```

8.8 MP4 录像

```
from media.sensor import *
from media.vencoder import *
from media.media import *
from media.mp4format import *

# 初始化sensor和vencoder（绑定方式）
mp4 = Mp4Container()
cfg = Mp4CfgStr(mp4.MP4_CONFIG_TYPE_MUXER)
cfg.SetMuxerCfg("test.mp4", mp4.MP4_CODEC_ID_H265, 1920, 1080, mp4.MP4_CODEC_ID_G711U)
mp4.Create(cfg); mp4.Start()
for i in range(200):
    mp4.Process()
mp4.Stop(); mp4.Destroy()
```

> **本文档涵盖立创·庐山派 K230-CanMV 开发板所有主要 API 模块和函数，所有内容已完整展开为内联说明，无需联网查阅。离线使用即可快速检索各函数和类的功能、参数及用法。**